



SciPy 進階求解方法

建構數位化工廠的數值運算引擎 (Unit 07 核心單元)

數位工具箱總覽



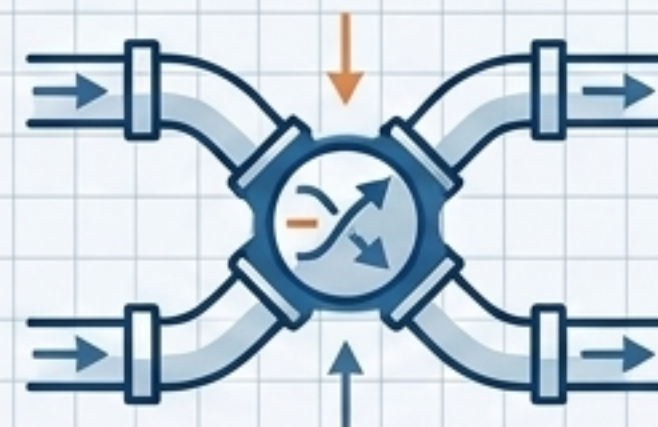
`root_scalar()`

單變數非線性方程式求解（統一介面）



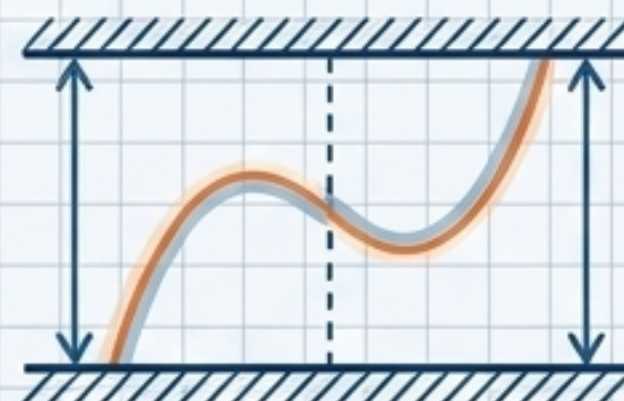
`root()`

多變數統一介面（支援擴展與大型系統）



`fsolve()`

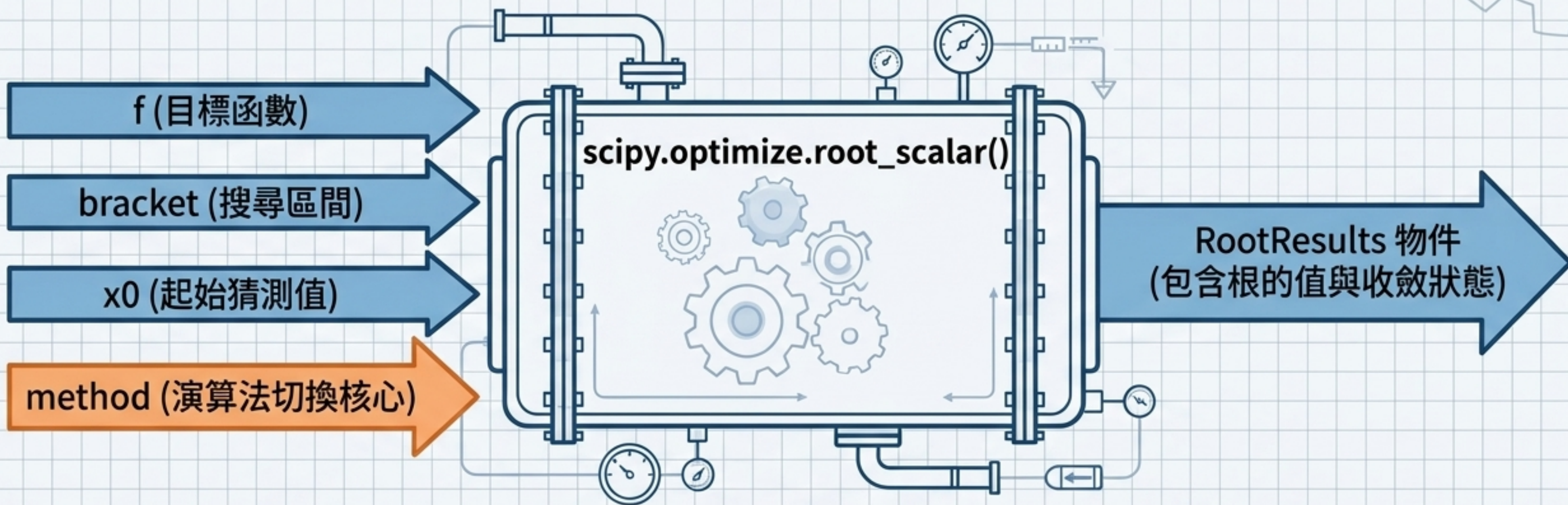
多變數方程組經典求解器（預設主力）



`least_squares()`

邊界約束與過確定系統（最佳化填補）

單變數求解器：root_scalar() 核心架構



```
sol = root_scalar(f, method='brentq', bracket=[a, b])
```



root_scalar 自 SciPy 1.2.0 起成為單變數問題的「統一介面」，透過 method 參數即可無縫切換底層引擎。

演算法評估矩陣 (Algorithm Comparison Matrix)

方法 (Method)	輸入需求 (Requirements)	收斂速度 (Speed)	實務推薦 (Recommendation)
brentq (Brent 方法)	bracket=[a, b]	中等 (超線性)	✅ 保證收斂，實務首選
newton (Newton-Raphson)	x0, fprime (導數)	極快 (二次收斂)	⚠️ 需提供解析導數，對 起始值敏感
secant (割線法)	x0, x1 (兩個猜測值)	快 (介於上述兩者間)	⚠️ 不需導數，但可能有 發散風險

化工實戰 (1) : Van der Waals 狀態方程式求解

$$\left(P + \frac{a}{V^2}\right)(V - b) - RT = 0$$

- 氣體 : CO₂
- 溫度 : T = 300 K
- 壓力 : P = 50 bar

brentq

迭代次數: 8 次

評價: 兼顧穩定與效率 (預設推薦)

newton

迭代次數: 4 次

評價: 速度最快, 但需算導數

secant

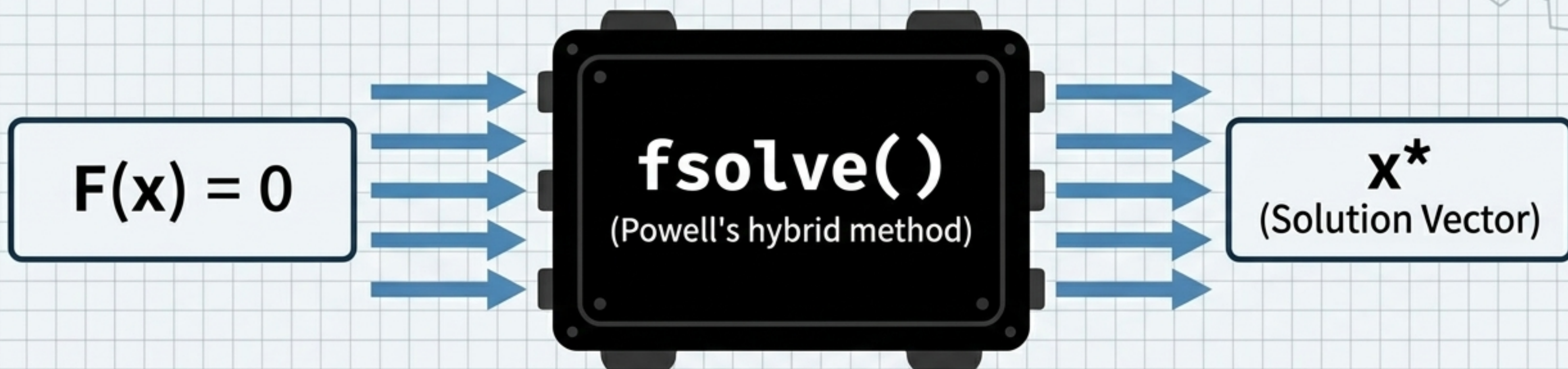
迭代次數: 5 次

評價: 免導數且快速, 但穩定較差



在物理可行域內, 三者皆精準收斂至 $V \approx 0.364 \text{ L/mol}$, 但 brentq 提供最具工程實用價值的穩定性。

多變數經典求解器：fsolve()



底層引擎

依賴極度穩定的 MINPACK 庫
(hybrd 演算法)。

工具地位

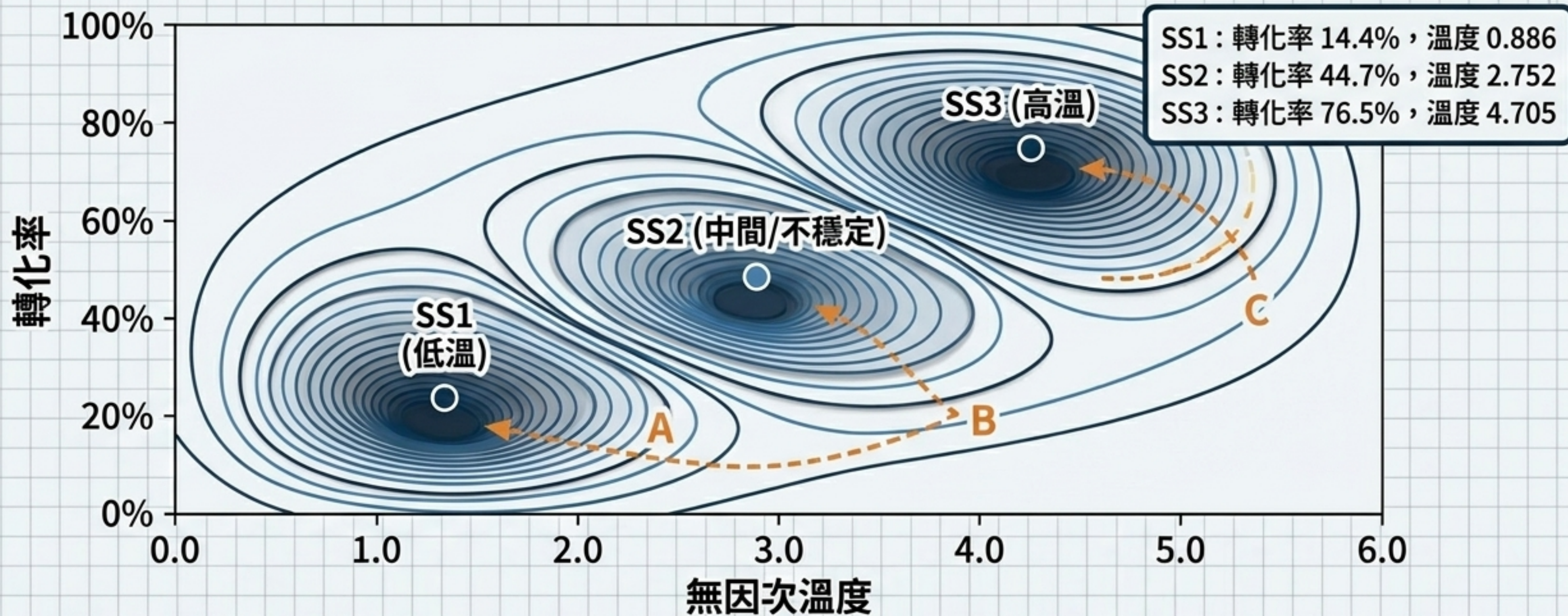
解決多變數非線性方程組的最
常用工具，多變數預設主力。

核心語法

```
sol = fsolve(func, x0)
```

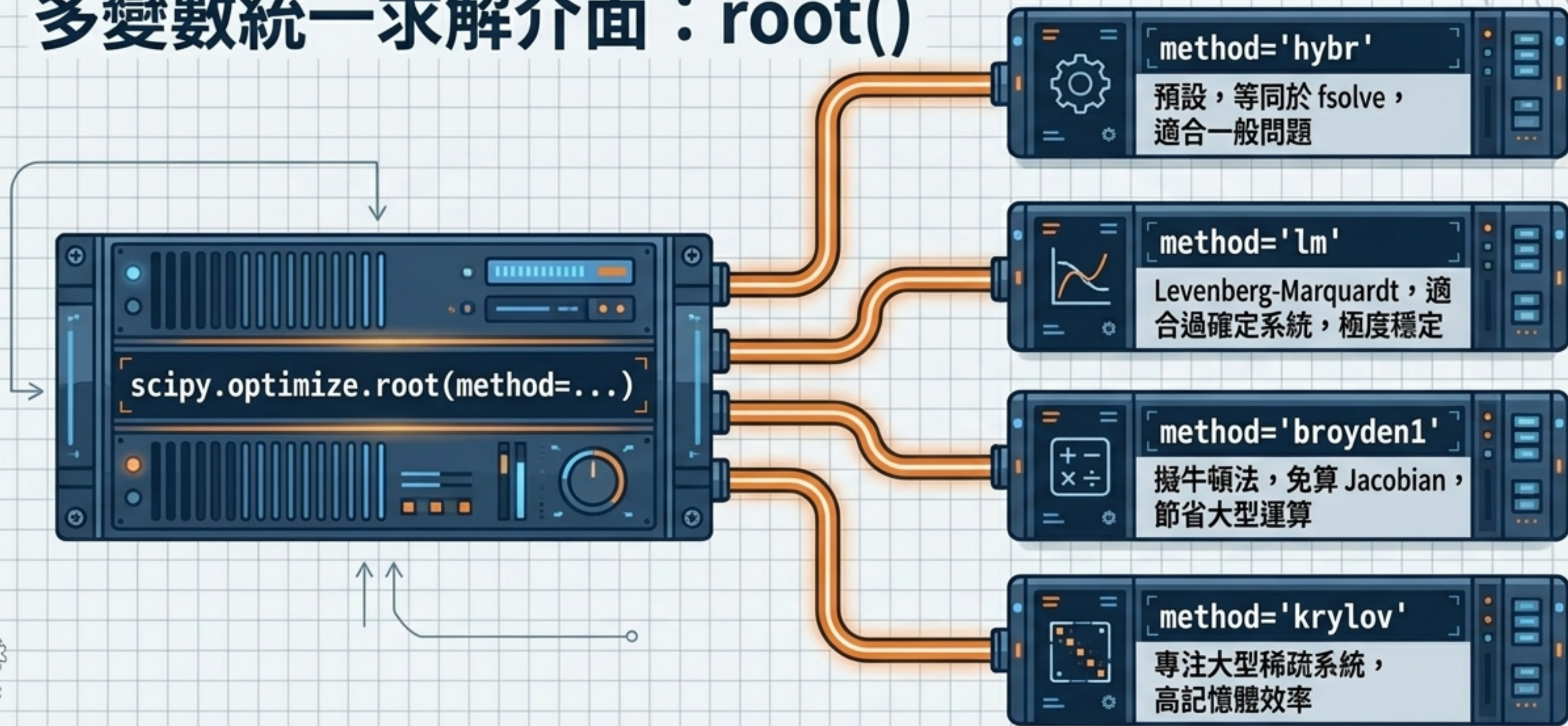
⚠ 警告：求解成功與否，高度依賴於起始猜測值 x_0 的選擇！

化工實戰 (2)：CSTR 多重穩態的探索



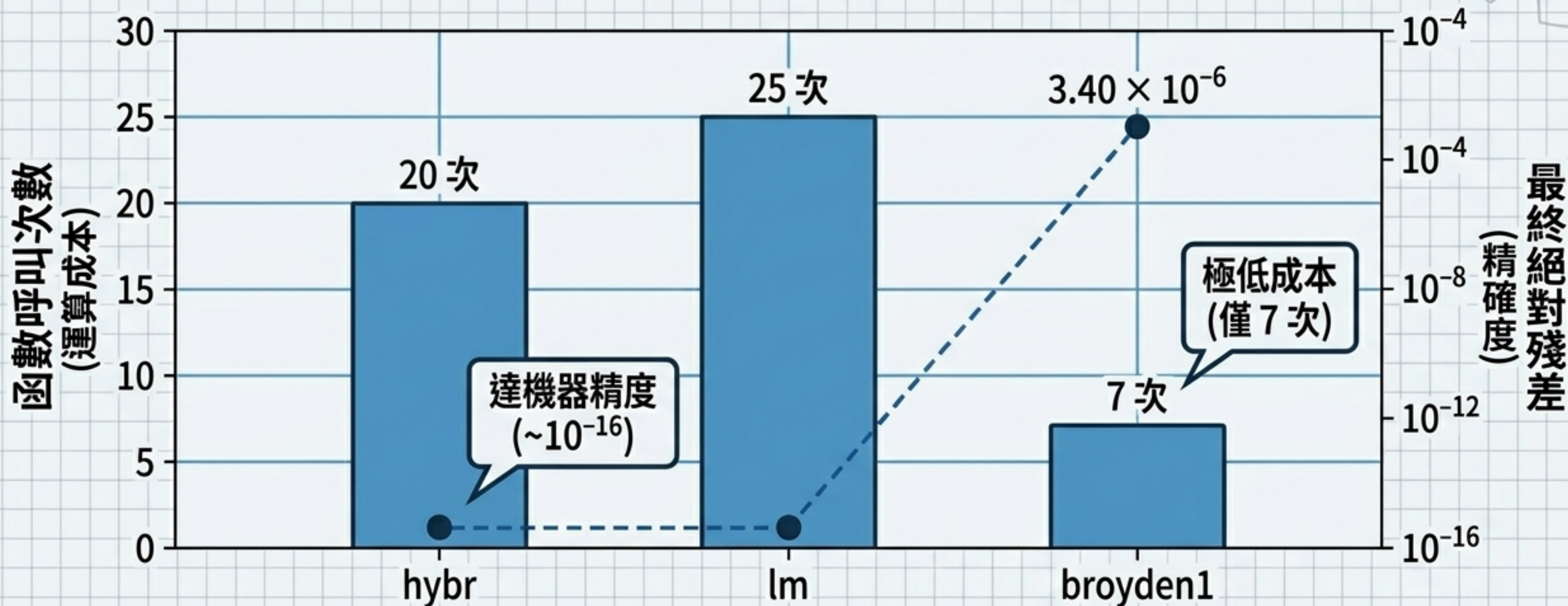
物理系統中的「多重穩態」意味著數學上的「多重根」。起始值 (x_0) 決定了演算法最終落入哪一個解的重力場。

多變數統一求解介面：root()



fsolve 是一個強大的單一工具，而 root 是整個演算法兵工廠的調度中心。
在面對極端規模或特殊幾何結構的方程組時，提供切換彈性。

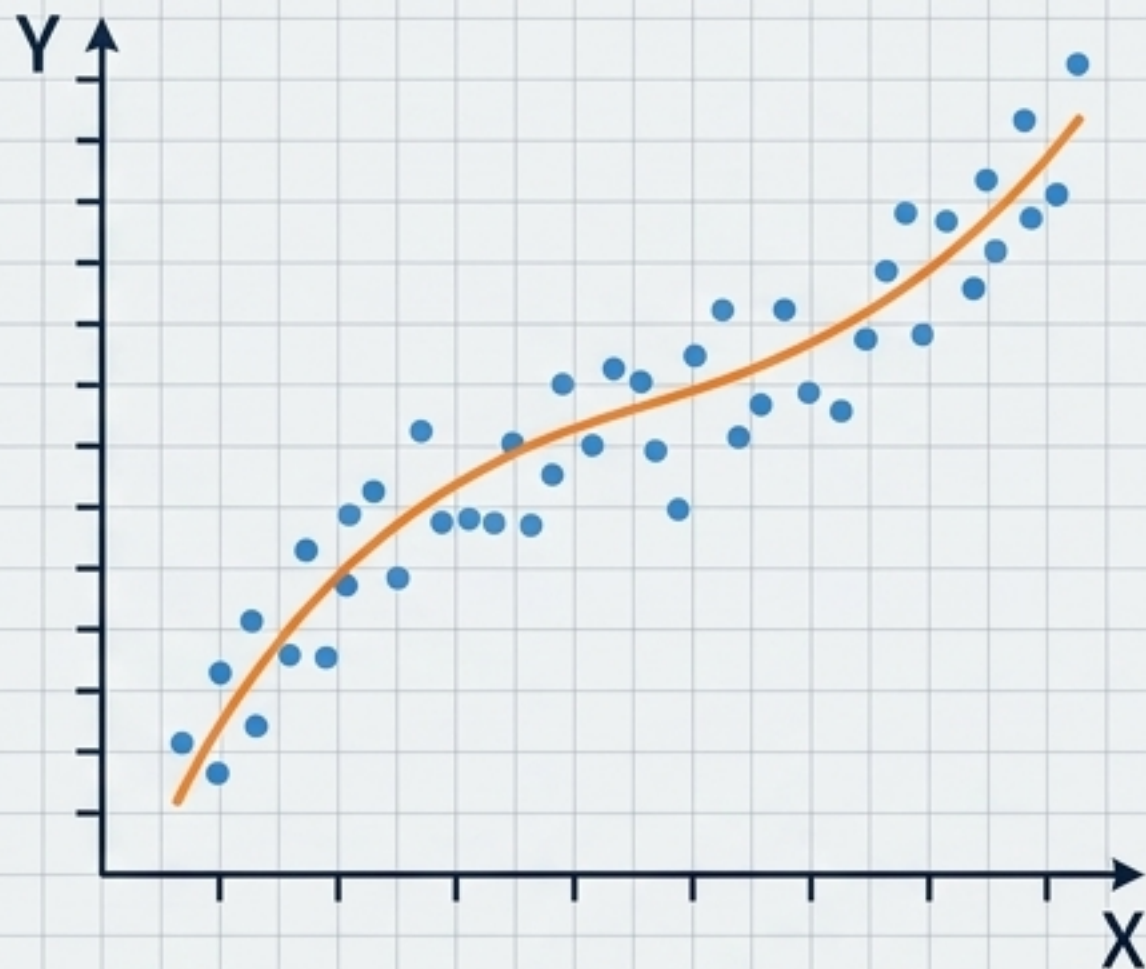
演算法效能對決 (Algorithm Shootout: CSTR)



⚠ 權衡 (Trade-off) : 擬牛頓法 (broyden1) 犧牲了極限精度來換取計算速度。對於高精度要求的化工模擬，hybr 或 lm 仍是首選。

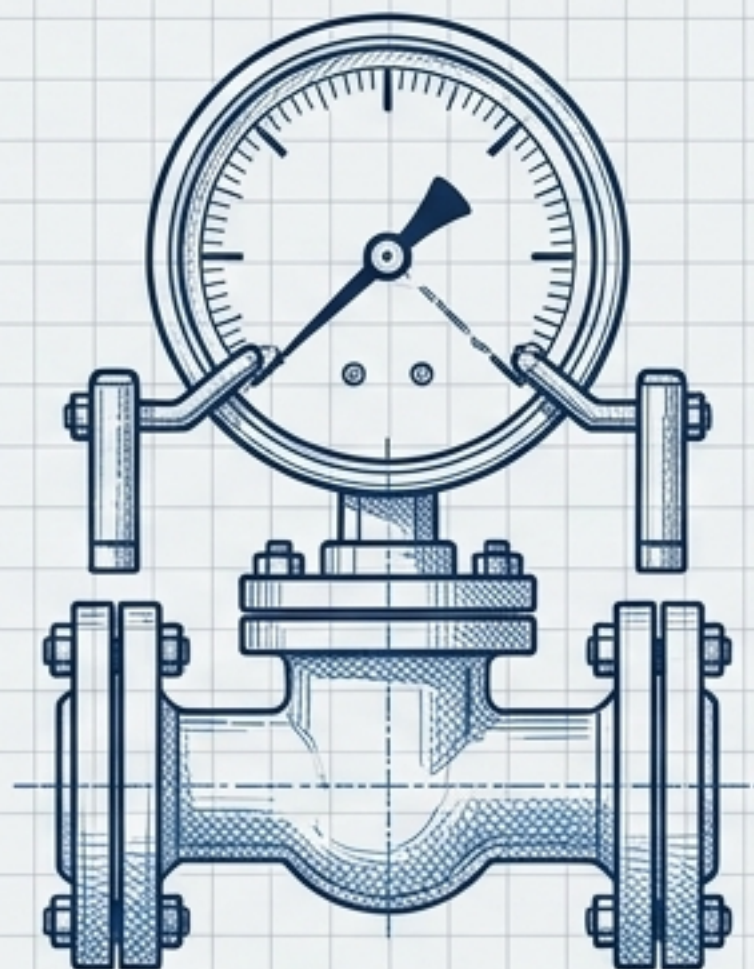
邊界約束與過確定系統：least_squares()

過確定系統 (Overdetermined)



當方程式數量多於未知數時，
尋找「最小化誤差」的最佳解。

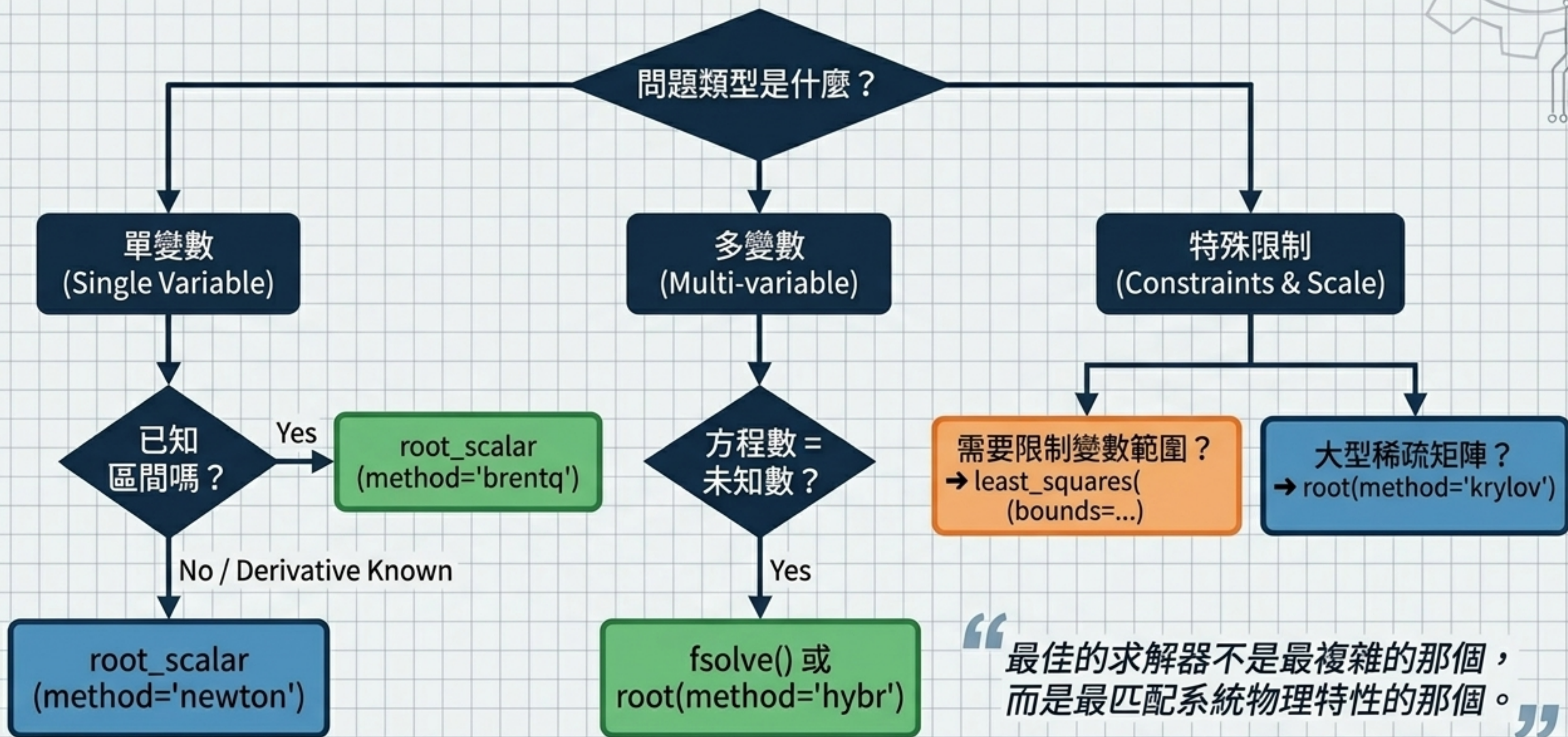
邊界約束 (Bounds)



避免求解過程跨入無物理意義的區域
(如負體積、超過 100% 的轉化率)。

```
least_squares(fun, x0, bounds=([low_limits], [high_limits]))
```


總結：求解器選擇指南 (Decision Tree)



“最佳的求解器不是最複雜的那個，
而是最匹配系統物理特性的那個。”